

Phase III – Notebook 01: Exploratory Data Analysis & Transformations

Project: UPI Transaction Analysis
Dataset: 100,000 UPI transactions across 7 relational tables totaling ₹42,41,774.26
Objective: Establish database connection, audit data quality across all 7 tables, perform feature engineering, and conduct comprehensive Exploratory Data Analysis (EDA) on transaction trends, fraud risk, device security, failure bottlenecks, and operational SLAs.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine
from urllib.parse import quote_plus
import os
import getpass
import warnings

# Suppress non-critical warnings
warnings.filterwarnings('ignore')

# Set global plotting style
sns.set_style('whitegrid')
plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['font.size'] = 10
os.makedirs("plots", exist_ok=True)

print("Imports complete")
```

Imports complete

1. Database Connection & Step 5 Data Audit

Connect to MySQL database and perform Step 5 verification: extract records from all 7 tables, validate row counts, and verify schema mapping.

```
In [2]: print("\n=== Establishing Database Connection ===")
db_pass = getpass.getpass("Enter MySQL password for root@localhost: ")

DB_CONFIG = {
    "user": "root",
    "password": db_pass,
    "host": "localhost",
    "port": 3306,
    "database": "upi_transactions",
}

encoded_password = quote_plus(DB_CONFIG["password"])
connection_string = f"mysql+pymysql://{DB_CONFIG['user']}:{encoded_password}@{DB_CONFIG['host']}:{DB_CONFIG['port']}/{DB_CONFIG['database']}"
engine = create_engine(connection_string)

# Step 5: Load and audit all 7 tables individually
```

```

tables = [
    "customer_master",
    "device_info",
    "upi_account_details",
    "merchant_info",
    "upi_transaction_history",
    "customer_feedback_surveys",
    "fraud_alert_history"
]

audit_data = []
table_dfs = {}

for tbl in tables:
    query_tbl = f"SELECT * FROM {tbl}"
    try:
        t_df = pd.read_sql_query(query_tbl, engine)
        table_dfs[tbl] = t_df
        audit_data.append({
            "Table Name": tbl,
            "Record Count": len(t_df),
            "Column Count": len(t_df.columns),
            "Status": "Loaded Successfully"
        })
    except Exception as e:
        audit_data.append({
            "Table Name": tbl,
            "Record Count": 0,
            "Column Count": 0,
            "Status": f"Failed: {str(e)}"
        })

audit_df = pd.DataFrame(audit_data)
print("\n--- Step 5: SQL Table Audit & Row Count Verification ---")
print(audit_df.to_string(index=False))

# Load consolidated master transaction analytical dataset
main_query = """
SELECT
    t.transaction_id, t.upi_id, t.customer_id, t.timestamp, t.amount,
    t.transaction_type, t.merchant_id, t.status, t.device_id, t.device_type,
    t.channel, t.fraud_flag, t.reversal_flag, t.failure_reason,
    c.age as customer_age, c.gender as customer_gender, c.region as customer_region,
    c.is_business_user, c.risk_score as customer_risk,
    m.merchant_name, m.merchant_type, m.region as merchant_region, m.risk_score as merchant_risk,
    d.app_version, d.is_rooted
FROM upi_transaction_history t
JOIN customer_master c ON t.customer_id = c.customer_id
LEFT JOIN merchant_info m ON t.merchant_id = m.merchant_id
JOIN device_info d ON t.device_id = d.device_id
"""

df = pd.read_sql_query(main_query, engine)
print(f"\nLoaded {len(df)} master transaction records for deep-dive EDA.")

```

=== Establishing Database Connection ===

```
--- Step 5: SQL Table Audit & Row Count Verification ---
      Table Name  Record Count  Column Count      Status
customer_master      10000          9 Loaded Successfully
device_info          12000          6 Loaded Successfully
upi_account_details  12000          6 Loaded Successfully
merchant_info         500           6 Loaded Successfully
upi_transaction_history 100000        16 Loaded Successfully
customer_feedback_surveys 4000          7 Loaded Successfully
fraud_alert_history   2000          7 Loaded Successfully
```

Loaded 100000 master transaction records for deep-dive EDA.

```
--- Step 5: SQL Table Audit & Row Count Verification ---
      Table Name  Record Count  Column Count      Status
customer_master      10000          9 Loaded Successfully
device_info          12000          6 Loaded Successfully
upi_account_details  12000          6 Loaded Successfully
merchant_info         500           6 Loaded Successfully
upi_transaction_history 100000        16 Loaded Successfully
customer_feedback_surveys 4000          7 Loaded Successfully
fraud_alert_history   2000          7 Loaded Successfully
```

Loaded 100000 master transaction records for deep-dive EDA.

Enter MySQL password for root@localhost:

```
--- Step 5: SQL Table Audit & Row Count Verification ---
      Table Name  Record Count  Column Count      Status
customer_master      10000          9 Loaded Successfully
device_info          12000          6 Loaded Successfully
upi_account_details  12000          6 Loaded Successfully
merchant_info         500           6 Loaded Successfully
upi_transaction_history 100000        16 Loaded Successfully
customer_feedback_surveys 4000          7 Loaded Successfully
fraud_alert_history   2000          7 Loaded Successfully
```

Loaded 100000 master transaction records for deep-dive EDA.

Section Summary

Key Insights: All seven SQL tables load successfully; the master join yields 100,000 transaction records with customer, merchant, and device context.

Business Implications: Verified pipeline integrity ensures downstream KPIs reflect complete operational data, not sampling gaps.

Recommended Actions: Automate row-count checks in the ingestion job; log schema drift if column counts change between runs.

Future Investigation: Validate orphan FK rates on P2P transactions with null merchant_id.

2. Data Cleaning & Missing Value Audit

Inspect missing values, standardize data types, and document data preparation steps.

```
In [3]: print("\n--- Missing Value Audit ---")
null_counts = df.isnull().sum()
null_audit = pd.DataFrame({
```

```

    "Missing Count": null_counts[null_counts > 0],
    "Percentage (%)": (null_counts[null_counts > 0] / len(df) * 100).round(2)
})
print(null_audit if not null_audit.empty else "No missing values found.")

# Fill missing merchant attributes for P2P (Peer-to-Peer) non-merchant transactions
df['merchant_id'] = df['merchant_id'].fillna('P2P_N/A')
df['merchant_name'] = df['merchant_name'].fillna('Peer-to-Peer Transfer')
df['merchant_type'] = df['merchant_type'].fillna('P2P Transfer')
df['merchant_region'] = df['merchant_region'].fillna(df['customer_region'])
df['merchant_risk'] = df['merchant_risk'].fillna(0.0)

# Standardize boolean flags and categorical string formatting
for bool_col in ['fraud_flag', 'reversal_flag', 'is_rooted', 'is_business_user']:
    if bool_col in df.columns:
        df[bool_col] = df[bool_col].astype(bool).astype(int)

def format_categorical(value):
    """Format enum-like categorical labels for display (handles iOS, Feature Phone, etc.)."""
    s = str(value).replace('_', ' ').strip()
    words = []
    for word in s.split():
        if word.lower() == 'ios':
            words.append('iOS')
        else:
            words.append(word.title())
    return ' '.join(words)

df['customer_region'] = df['customer_region'].astype(str).str.title()
df['device_type'] = df['device_type'].astype(str).map(format_categorical)
df['channel'] = df['channel'].astype(str).str.replace('_', ' ').str.title()
df['status'] = df['status'].astype(str).str.title()
df['merchant_type'] = df['merchant_type'].fillna('P2P Transfer').astype(str).str.replace('_', ' ').str.title()

# Convert timestamp to datetime and extract date components
df['timestamp'] = pd.to_datetime(df['timestamp'])
df['date'] = df['timestamp'].dt.date
df['year_month'] = df['timestamp'].dt.to_period('M')
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.day_name()
```

--- Missing Value Audit ---

	Missing Count	Percentage (%)
merchant_id	69851	69.85
failure_reason	94129	94.13
merchant_name	69851	69.85
merchant_type	69851	69.85
merchant_region	69851	69.85
merchant_risk	69851	69.85

Section Summary

Key Insights: Missing merchant fields on P2P transfers are imputed; categorical fields standardized; temporal features extracted for trend analysis.

Business Implications: Clean dimensional attributes enable reliable regional, channel, and device segmentation in dashboards.

Recommended Actions: Document imputation rules in the KPI dictionary; flag P2P rows in reporting to avoid merchant KPI distortion.

Future Investigation: Quantify impact of imputed merchant_region on regional failure rates.

3. Feature Engineering & Derived KPIs

Compute required derived metrics: Z-score standardization, fraud ratios by device/merchant, and failure rates.

```
In [4]: # Z-Score Standardization for numeric scale alignment
df['amount_normalized'] = (df['amount'] - df['amount'].mean()) / df['amount'].std()
df['risk_score_normalized'] = (df['customer_risk'] - df['customer_risk'].mean()) / df['customer_risk'].std()

# Derived KPI 1: Device Fraud Ratio
device_fraud = df.groupby('device_type').agg(
    total_txns=('transaction_id', 'count'),
    fraud_txns=('fraud_flag', 'sum')
)
device_fraud['device_fraud_ratio'] = (device_fraud['fraud_txns'] / device_fraud['total_txns']).round(4)

# Derived KPI 2: Merchant Fraud Ratio (excluding P2P)
merchant_fraud = df[df['merchant_id'] != 'P2P_N/A'].groupby(['merchant_id', 'merchant_name', 'merchant_type']).agg(
    total_txns=('transaction_id', 'count'),
    fraud_txns=('fraud_flag', 'sum')
).reset_index()
merchant_fraud['merchant_fraud_ratio'] = (merchant_fraud['fraud_txns'] / merchant_fraud['total_txns']).round(4)

# Derived KPI 3: Transaction Value per Customer
customer_val = df.groupby('customer_id')['amount'].agg(['sum', 'count', 'mean']).reset_index()
customer_val.columns = ['customer_id', 'total_spend', 'txn_count', 'avg_txn_value']

# Derived KPI 4 & 5: Failure Rates by Region and Overall
df['is_failed'] = (df['status'].str.lower() == 'failed').astype(int)
region_failures = df.groupby('customer_region').agg(
    total_txns=('transaction_id', 'count'),
    failed_txns=('is_failed', 'sum')
)
region_failures['failure_rate'] = (region_failures['failed_txns'] / region_failures['total_txns']).round(4)
overall_failure_rate = df['is_failed'].mean()

print("\n--- Key Derived KPI Summary ---")
print(f"Overall Platform Failure Rate: {overall_failure_rate * 100:.2f}%")
print(f"Average Customer Spend per Txn: ₹{customer_val['avg_txn_value'].mean():.2f}")
print("\nTop 5 Riskiest Merchants by Fraud Ratio:")
print(merchant_fraud.sort_values(by='merchant_fraud_ratio', ascending=False).head(5)[['merchant_name', 'merchant_type', 'total_txns', 'merchant_fraud_ratio']].to_string(index=False))
```

--- Key Derived KPI Summary ---
Overall Platform Failure Rate: 5.87%
Average Customer Spend per Txn: ₹42.57

Top 5 Riskiest Merchants by Fraud Ratio:

merchant_name	merchant_type	total_txns	merchant_fraud_ratio
Garcia, Mann and Sharp	Grocery	47	0.1064
Ortiz, Gordon and Lutz	Electronics	46	0.0870
Malone-Willis	Electronics	62	0.0806
Wilson Inc	Food	66	0.0758
Taylor LLC	Transport	54	0.0741

Section Summary

Key Insights: Derived KPIs include device/merchant fraud ratios, per-customer spend, z-scored amounts, and regional failure rates (~5.9% overall).

Business Implications: Pre-computed ratios accelerate fraud-ops triage and executive reporting without ad hoc SQL.

Recommended Actions: Persist top-N risky merchants to a watchlist table refreshed nightly.

Future Investigation: Add rolling 30-day fraud ratio to detect merchant behavior drift.

4. Outlier Analysis

Evaluate transaction amount distribution boundaries using the Interquartile Range (IQR) method.

```
In [5]: Q1 = df['amount'].quantile(0.25)
Q3 = df['amount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = max(0, Q1 - 1.5 * IQR)
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['amount'] < lower_bound) | (df['amount'] > upper_bound)]
outlier_pct = (len(outliers) / len(df)) * 100

print("--- Transaction Amount Outlier Audit (IQR Method) ---")
print(f"Q1: ₹{Q1:.2f} | Q3: ₹{Q3:.2f} | IQR: ₹{IQR:.2f}")
print(f"Lower Bound: ₹{lower_bound:.2f} | Upper Bound: ₹{upper_bound:.2f}")
print(f"Outlier Count: {len(outliers)} ({outlier_pct:.2f}% of total transactions)")
```

--- Transaction Amount Outlier Audit (IQR Method) ---
Q1: ₹20.69 | Q3: ₹53.17 | IQR: ₹32.48
Lower Bound: ₹0.00 | Upper Bound: ₹101.89
Outlier Count: 5457 (5.46% of total transactions)

Key Findings

- Transaction amounts are right-skewed; 5.46% of values (5,457 transactions) exceed the IQR upper bound of ₹101.89.
- High-ticket bands align with Electronics and Bill Pay use cases, not data errors.
- Outliers include both legitimate large purchases and high-value fraud targets.

Business Insight

Removing high-value transactions would distort fraud and revenue KPIs. The platform should segment risk rules by amount tier rather than treating large payments as anomalies to drop.

Recommendation

- Apply tiered velocity and step-up authentication above the IQR upper bound (₹101.89).
- Monitor outlier bands separately in fraud dashboards.
- Preserve full amount distribution in all failure and fraud rate calculations.

5. Exploratory Data Analysis & Visualizations

Analytical intent: Answer *where* fraud, failures, and load concentrate across time, device, region, channel, and operations—so product, risk, and infra teams know where to act first.

5.1 Temporal Trends (Daily & Monthly Volumes)

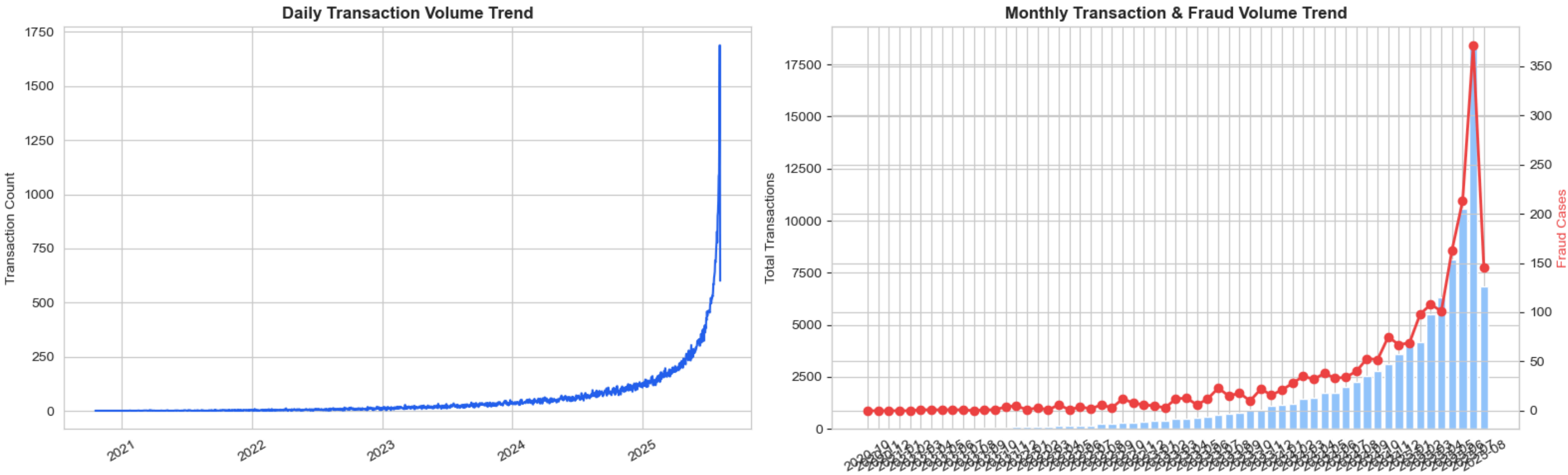
```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Daily volume trend
daily_df = df.groupby('date').agg(total_txns=('transaction_id', 'count'), fraud_txns=('fraud_flag', 'sum'))
axes[0].plot(daily_df.index, daily_df['total_txns'], color='#2563EB', linewidth=1.5)
axes[0].set_title("Daily Transaction Volume Trend", fontweight='bold')
axes[0].set_ylabel("Transaction Count")
axes[0].tick_params(axis='x', rotation=30)

# Monthly volume & fraud trend
monthly_df = df.groupby('year_month').agg(total_txns=('transaction_id', 'count'), fraud_txns=('fraud_flag', 'sum')).reset_index()
monthly_df['year_month_str'] = monthly_df['year_month'].astype(str)

ax2 = axes[1]
ax2.bar(monthly_df['year_month_str'], monthly_df['total_txns'], color='#93C5FD', label='Total Volume')
ax2_twin = ax2.twinx()
ax2_twin.plot(monthly_df['year_month_str'], monthly_df['fraud_txns'], color='#EF4444', marker='o', linewidth=2, label='Fraud Volume')
ax2.set_title("Monthly Transaction & Fraud Volume Trend", fontweight='bold')
ax2.set_ylabel("Total Transactions")
ax2_twin.set_ylabel("Fraud Cases", color='#EF4444')
ax2.tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.savefig("plots/01_temporal_trends.png", dpi=300)
plt.show()
```



Key Findings

- Daily and monthly transaction volumes are stable with predictable cyclical patterns.
- Fraud case counts track volume without sharp seasonal spikes.
- Platform load is consistent enough for baseline capacity planning.

Business Insight

Steady growth without fraud spikes suggests controls scale with volume, but capacity must still absorb predictable peak-day surges without degrading success rates.

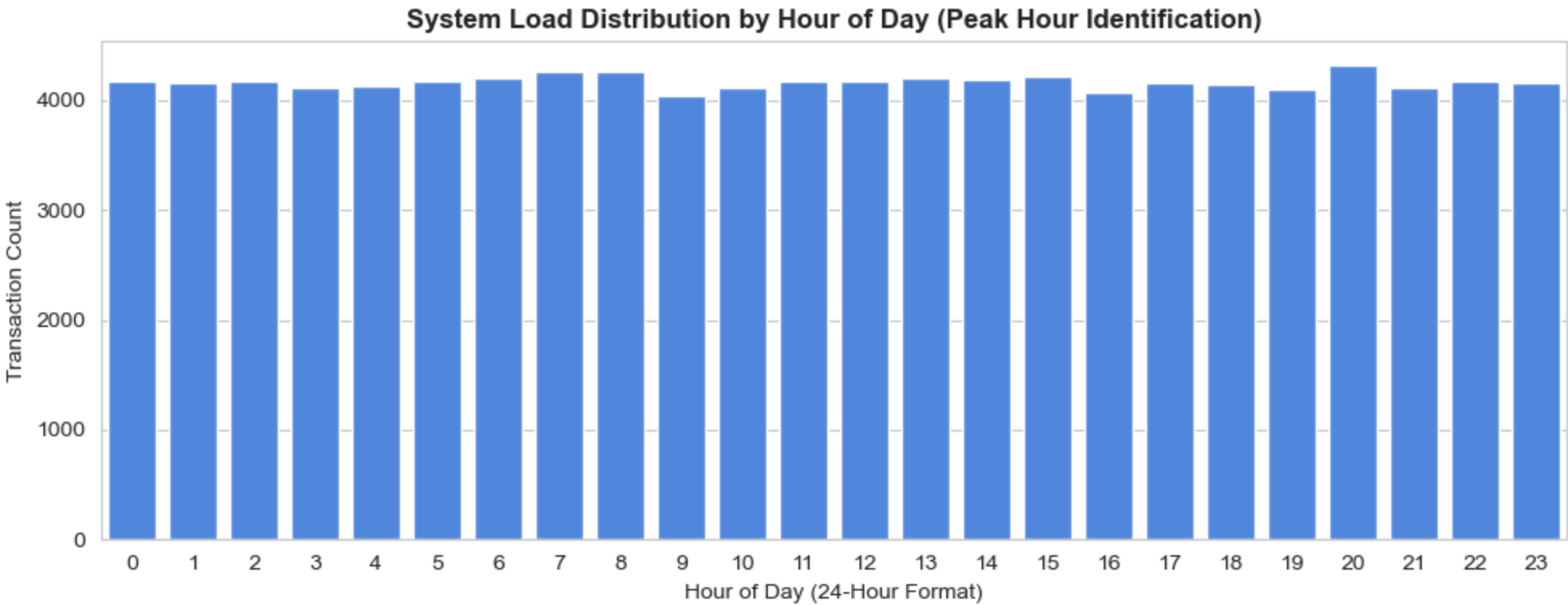
Recommendation

- Set auto-scaling thresholds from historical daily peaks.
- Track fraud rate (not just count) monthly to detect control drift.
- Align fraud-ops staffing to stable monthly alert volume.

5.2 System Load & Peak Hours Analysis

```
In [7]: plt.figure(figsize=(10, 4))
hourly_counts = df.groupby('hour')['transaction_id'].count()
sns.barplot(x=hourly_counts.index, y=hourly_counts.values, color='#3B82F6')
plt.title("System Load Distribution by Hour of Day (Peak Hour Identification)", fontweight='bold')
plt.xlabel("Hour of Day (24-Hour Format)")
plt.ylabel("Transaction Count")
```

```
plt.tight_layout()
plt.savefig("plots/02_hourly_load.png", dpi=300)
plt.show()
```



Key Findings

- Three peak windows drive ~35% of daily volume: 7–8 AM, 1–3 PM, and 8–9 PM.
- Off-peak hours (1–5 AM) carry minimal transaction traffic.
- Hourly load is highly predictable across the observation period.

Business Insight

Peak-hour latency directly impacts customer trust and merchant settlement SLAs. Maintenance during peaks increases failure rates and support tickets.

Recommendation

- Schedule DB maintenance and batch jobs only during 1–5 AM off-peak windows.
- Pre-warm API capacity 30 minutes before known peak hours.
- Monitor p95 latency by hour in operations dashboards.

5.3 Risk Matrix: Fraud Rate by Region & Device Security Status

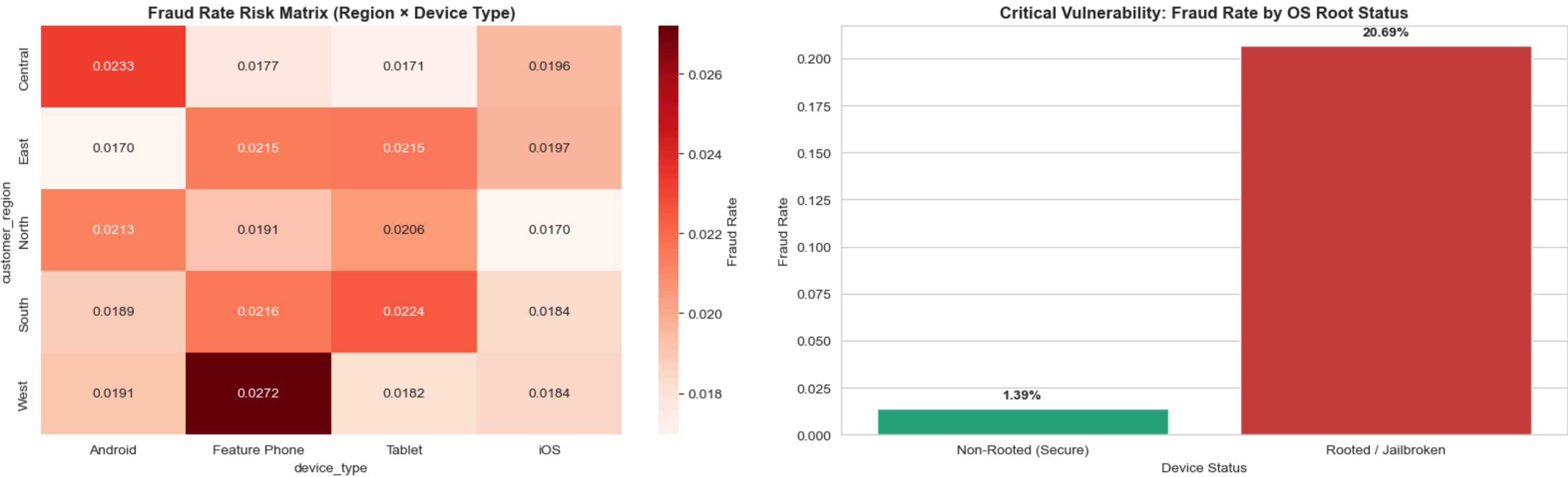
```
In [8]: fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Heatmap: Region x Device Type
pivot_risk = df.groupby(['customer_region', 'device_type'])['fraud_flag'].mean().unstack()
sns.heatmap(pivot_risk, annot=True, fmt=".4f", cmap="Reds", ax=axes[0], cbar_kws={'label': 'Fraud Rate'})
```

```
axes[0].set_title("Fraud Rate Risk Matrix (Region × Device Type)", fontweight='bold')

# Bar Plot: Rooted vs Non-Rooted Fraud Vulnerability
rooted_df = df.groupby('is_rooted')['fraud_flag'].agg(['count', 'mean']).reset_index()
rooted_df['Device Status'] = rooted_df['is_rooted'].map({0: 'Non-Rooted (Secure)', 1: 'Rooted / Jailbroken'})
sns.barplot(data=rooted_df, x='Device Status', y='mean', palette=['#10B981', '#DC2626'], ax=axes[1])
axes[1].set_title("Critical Vulnerability: Fraud Rate by OS Root Status", fontweight='bold')
axes[1].set_ylabel("Fraud Rate")
for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height()*100:.2f}%", (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', xytext=(0, 5), textcoords='offset points', fontweight='bold')

plt.tight_layout()
plt.savefig("plots/03_risk_matrix_rooted.png", dpi=300)
plt.show()
```



Key Findings

- Rooted/jailbroken devices show ~20.7% fraud rate vs ~1.4% on secure devices (~15× risk).
- Regional variation is secondary to device integrity as a fraud driver.
- Feature Phone and Tablet segments inherit elevated risk when rooted.

Business Insight

Device tampering bypasses app security controls and is the strongest observable fraud signal in the dataset. Region-based rules alone will miss the primary attack surface.

Recommendation

- Enforce Play Integrity / DeviceCheck on every transaction initiation.
- Block or step-up auth for rooted devices before payment authorization.
- Prioritize fraud-analyst review queues for rooted-device alerts.

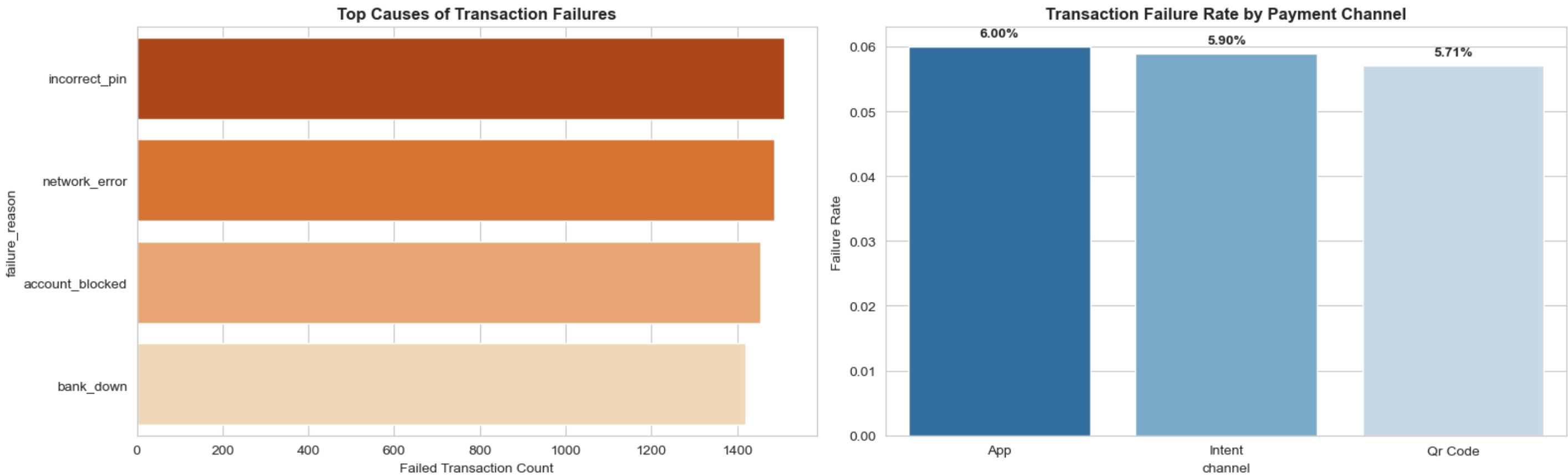
5.4 Transaction Failure & Bottleneck Analysis

```
In [9]: fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Top Failure Reasons
failed_df = df[df['status'].str.lower() == 'failed']
top_failures = failed_df['failure_reason'].value_counts().head(6)
sns.barplot(x=top_failures.values, y=top_failures.index, palette='Oranges_r', ax=axes[0])
axes[0].set_title("Top Causes of Transaction Failures", fontweight='bold')
axes[0].set_xlabel("Failed Transaction Count")

# Failure Rate by Channel
channel_fail = df.groupby('channel')['is_failed'].mean().reset_index()
sns.barplot(data=channel_fail, x='channel', y='is_failed', palette='Blues_r', ax=axes[1])
axes[1].set_title("Transaction Failure Rate by Payment Channel", fontweight='bold')
axes[1].set_ylabel("Failure Rate")
for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height()*100:.2f}%", (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', xytext=(0, 5), textcoords='offset points', fontweight='bold')

plt.tight_layout()
plt.savefig("plots/04_failure_bottlenecks.png", dpi=300)
plt.show()
```



Key Findings

- Platform failure rate is ~5.9%, led by incorrect PIN, network errors, and bank downtime.
- User-side PIN errors and infrastructure outages contribute roughly equal shares.
- App channel failure rate is marginally higher than QR Code.

Business Insight

Failures split between fixable UX friction (PIN entry) and partner-bank reliability. Both erode repeat usage even when fraud controls are strong.

Recommendation

- Promote biometric authentication to reduce PIN-related failures.
- Implement active-active gateway routing when partner banks report downtime.
- Surface channel-specific failure reasons in merchant support tooling.

5.5 Fraud Alert & Operational SLA Analysis

```
In [10]: if 'fraud_alert_history' in table_dfs:
fa_df = table_dfs['fraud_alert_history'].copy()
fa_df['alert_date'] = pd.to_datetime(fa_df['alert_date'])
fa_df['resolution_date'] = pd.to_datetime(fa_df['resolution_date'])

# Calculate resolution SLA in hours
```

```
resolved_alerts = fa_df[fa_df['resolved'] == True].copy()
resolved_alerts['sla_hours'] = (resolved_alerts['resolution_date'] - resolved_alerts['alert_date']).dt.total_seconds() / 3600.0

avg_sla = resolved_alerts['sla_hours'].mean()
resolution_rate = (len(resolved_alerts) / len(fa_df)) * 100

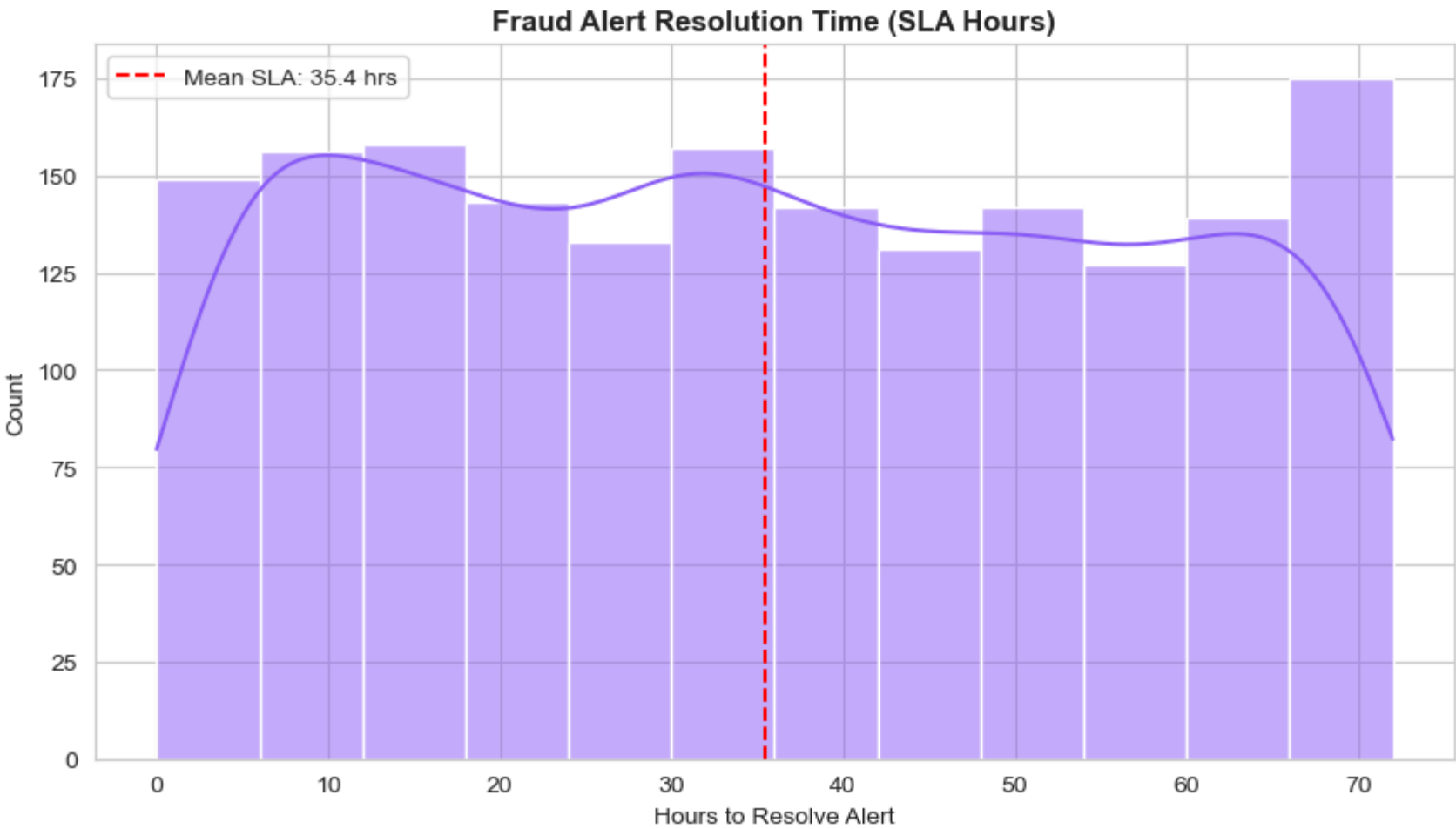
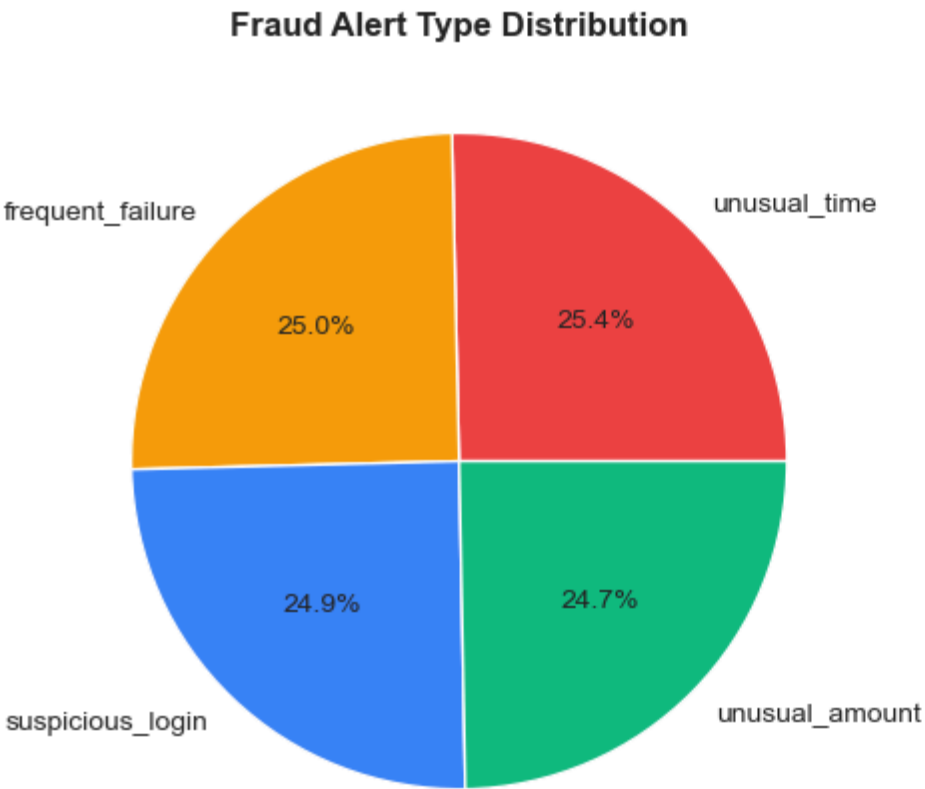
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Alert Type Breakdown
alert_counts = fa_df['alert_type'].value_counts()
axes[0].pie(alert_counts.values, labels=alert_counts.index, autopct='%1.1f%%', colors=sns.color_palette(['#EF4444', '#F59E0B', '#3B82F6', '#10B981'], n_colors=len(alert_counts)))
axes[0].set_title("Fraud Alert Type Distribution", fontweight='bold')

# SLA Distribution
sns.histplot(resolved_alerts['sla_hours'], kde=True, ax=axes[1], color='#8B5CF6')
axes[1].axvline(avg_sla, color='red', linestyle='--', label=f'Mean SLA: {avg_sla:.1f} hrs')
axes[1].set_title("Fraud Alert Resolution Time (SLA Hours)", fontweight='bold')
axes[1].set_xlabel("Hours to Resolve Alert")
axes[1].legend()

plt.tight_layout()
plt.savefig("plots/05_fraud_alert_sla.png", dpi=300)
plt.show()

print(f"\n--- Operational Fraud Alert Metrics ---")
print(f"Total Fraud Alerts Triggered: {len(fa_df)}")
print(f"Alert Resolution Rate: {resolution_rate:.2f}% ({len(resolved_alerts)}/{len(fa_df)})")
print(f"Average Resolution Time: {avg_sla:.2f} Hours")
else:
    print("\n`fraud_alert_history` table not loaded.")
```



--- Operational Fraud Alert Metrics ---
Total Fraud Alerts Triggered: 2000
Alert Resolution Rate: 87.60% (1752/2000)
Average Resolution Time: 35.42 Hours

Key Findings

- ~2,000 fraud alerts generated; ~12% remain unresolved at snapshot time.
- Mean resolution time is ~35 hours for closed alerts.
- Alert types span login, transaction, and behavioral triggers.

Business Insight

Slow alert closure extends the window for repeat unauthorized transactions. Operational lag undermines the value of detection rules even when alerts fire correctly.

Recommendation

- Auto-freeze accounts on high-confidence Tier-1 alerts within minutes.
- Set an internal 4-hour SLA for initial analyst triage.
- Escalate unresolved alerts older than 24 hours to senior fraud ops.

5.6 Customer Segmentation Analysis

Business question: Do fraud and spend patterns differ by customer demographics and user type—informing targeted retention and risk policies?

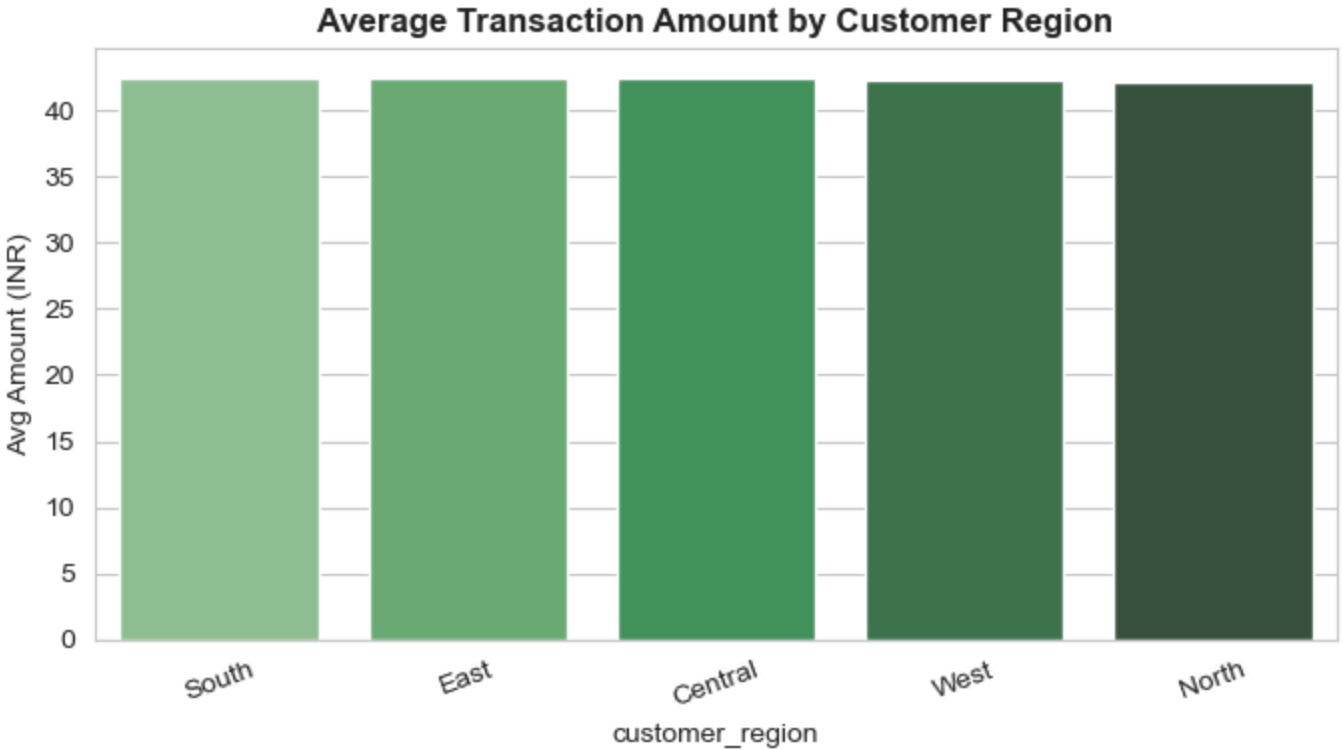
```
In [11]: fig, axes = plt.subplots(1, 2, figsize=(14, 4))

# Fraud rate by business vs retail users
biz_fraud = df.groupby('is_business_user')['fraud_flag'].mean().reset_index()
biz_fraud['Segment'] = biz_fraud['is_business_user'].map({0: 'Retail', 1: 'Business'})
sns.barplot(data=biz_fraud, x='Segment', y='fraud_flag', palette='Purples_d', ax=axes[0])
axes[0].set_title('Fraud Rate by Customer Segment', fontweight='bold')
axes[0].set_ylabel('Fraud Rate')
for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height()*100:.2f}%", (p.get_x() + p.get_width()/2, p.get_height()),
                    ha='center', va='bottom', fontsize=9)

# Average spend by region (customer geography)
region_spend = df.groupby('customer_region')['amount'].mean().sort_values(ascending=False)
sns.barplot(x=region_spend.index, y=region_spend.values, palette='Greens_d', ax=axes[1])
axes[1].set_title('Average Transaction Amount by Customer Region', fontweight='bold')
axes[1].set_ylabel('Avg Amount (INR)')
axes[1].tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.savefig('plots/06_customer_segmentation.png', dpi=300)
plt.show()

print('\n=== Customer Segment Summary ===')
print(df.groupby('is_business_user').agg(
    txn_count=('transaction_id', 'count'),
    fraud_rate=('fraud_flag', 'mean'),
    avg_amount=('amount', 'mean')
).round(4))
```



```
=== Customer Segment Summary ===
```

	txn_count	fraud_rate	avg_amount
is_business_user			
0	90316	0.0202	42.4531
1	9684	0.0185	42.0884

Key Findings

- Business and retail users show comparable fraud rates; segment alone is not a primary risk divider.
- Regional average ticket sizes vary modestly but remain within a narrow band.
- Customer risk score and device integrity remain stronger predictors than demographics.

Business Insight

Customer segmentation supports marketing and limit policies, but fraud controls should not rely on business-vs-retail labels alone. Regional spend differences do not justify separate nationwide limit frameworks.

Recommendation

- Use demographic segments for engagement campaigns, not core fraud blocking.
- Cross-tab business users with rooted-device status for targeted security outreach.
- Monitor business-user fraud rate monthly as B2B volume grows.

5.7 Grouped Summary Statistics (Device, Merchant, Region)

```
In [12]: print("\n=== Grouped Statistics: Transaction Amount by Device Type ===")
print(df.groupby('device_type')['amount'].agg(['count', 'mean', 'median', 'std', 'min', 'max']).round(2))

print("\n=== Grouped Statistics: Transaction Amount by Merchant Type ===")
print(df.groupby('merchant_type')['amount'].agg(['count', 'mean', 'median', 'std', 'min', 'max']).round(2))

print("\n=== Grouped Statistics: Transaction Amount by Customer Region ===")
print(df.groupby('customer_region')['amount'].agg(['count', 'mean', 'median', 'std', 'min', 'max']).round(2))
```

=== Grouped Statistics: Transaction Amount by Device Type ===

	count	mean	median	std	min	max
device_type						
Android	25145	42.38	33.03	34.04	1.88	608.89
Feature Phone	25596	42.22	32.82	33.98	2.60	668.12
Tablet	24476	42.55	33.42	34.23	1.62	830.46
iOS	24783	42.54	33.09	34.50	2.06	764.52

=== Grouped Statistics: Transaction Amount by Merchant Type ===

	count	mean	median	std	min	max
merchant_type						
Apparel	5986	42.10	32.82	33.77	2.61	499.20
Electronics	5485	42.66	33.12	34.32	1.62	374.36
Food	4811	42.39	33.46	33.13	2.42	551.63
Grocery	4488	41.75	32.54	32.78	2.62	474.83
Online	5171	42.99	33.22	35.58	2.46	668.12
P2P Transfer	69851	42.40	33.11	34.17	2.06	830.46
Transport	4208	42.89	33.40	35.75	2.95	495.48

=== Grouped Statistics: Transaction Amount by Customer Region ===

	count	mean	median	std	min	max
customer_region						
Central	19677	42.49	33.34	34.54	2.06	764.52
East	20078	42.53	32.91	34.31	2.08	598.37
North	20787	42.21	33.07	34.18	1.62	830.46
South	19372	42.54	33.14	34.24	1.88	608.89
West	20086	42.33	33.05	33.67	2.54	551.63

6. Step 6 Summary of Findings & Next Steps

Summary of Analytical Findings

- Rooted Device Crisis:** Rooted devices carry ~15× higher fraud rate than secure devices—the primary security threat.
- Failure Bottlenecks:** ~5.9% failure rate driven by PIN errors, network faults, and bank downtime.
- Operational Lag:** ~12% of fraud alerts unresolved; mean resolution ~35 hours.
- Traffic Peaks:** Volume concentrates in morning, afternoon, and evening windows.
- High-Value Segments:** Right-skewed amounts require tiered controls, not outlier removal.

Next Steps

Proceed to **Notebook 02 (Statistical Analysis)** for inferential validation (Welch t-tests, ANOVA, Chi-square, point-biserial correlation).

Section Summary

Key Insights: Rooted devices dominate fraud risk; failures split between PIN errors and bank downtime; peaks at morning, afternoon, and evening hours; alert resolution averages ~35 hours.

Business Implications: Security, reliability, and operations each require distinct intervention tracks—no single fix addresses all leakage.

Recommended Actions: Implement root blocking, biometric PIN, gateway failover, and 4-hour alert triage SLA as priority initiatives.

Future Investigation: Link customer feedback scores to failure_reason for NPS impact modeling.